End-to-end system solution for the Internet of Things

IoT Final Project 2026-27

Reconnecting with Nature

Groups: 2–3 persons. Keep the same groups as in the IoT Challenge.

Weight: 65% of the final grade.

Submission: Friday, 11 December 2026, through ALUD.

Demonstration: week of 14 December 2026, in person.

1. The objective

This is the final project of the course and the continuation of your IoT Challenge. You are asked to implement the idea you proposed in the challenge, within this year's theme: "IoT for Reconnecting with Nature." The implementation may differ from your initial proposal, either because you think your objectives fell short or because you were too ambitious. That is fine, as long as the core concept and its value for the people you identified remain recognisable.

The overall idea is that you put into practice as many of the concepts seen throughout the course as possible, starting from what you have built in the labs: the Raspberry Pi 5, the Grove sensors in Python, your own HTTP server and the data publishing pipeline.

2. What will be assessed

2.1 Complexity and scope

- The work should be commensurate with the size of the group, the weight of the project in the subject and the time available. Be realistic about the effort you can devote to it alongside the workload of other subjects.

2.2 Variety of sensors and actuators, and their role in the project

- Use sensors of different nature (digital GPIO, analogue, I2C).
- Relate sensor inputs and actuator outputs coherently.
- Make sense of the data: establish qualitative levels to represent it (e.g., "good / fair / poor conditions for a walk").
- Perform simple analytics on the data, for example a report based on averages, medians or trends.
- Create alerts or actions with the available actuators (if something happens, then trigger them).

2.3 Data storage and visualisation

- Use any of the options seen in the final lab: InfluxDB + Grafana (on the Raspberry Pi, on your laptop or hybrid), ThingsBoard[1] or ThingSpeak.
- You may also deploy the database or dashboard with Docker containers, or use any other database, IoT platform or visualisation tool you are familiar with (e.g., AWS IoT, Azure IoT).

---

[1]ThingsBoard with Grove sensors on Raspberry Pi:
https://thingsboard.io/docs/samples/raspberry/grove/

- Use JSON for data delivery; other formats (e.g., XML, CBOR) are also acceptable if you justify them.

2.4 Originality in the use of additional tools

- Use frameworks not seen in class or integrate other tools you know.
- Build an application with a web interface to interact with the actuators or view their status, extending your lab HTTP server: Flask[2], FastAPI, React, Node.js, Node-RED[3], Django… On the Raspberry Pi 5 use the Grove library or gpiozero[4] for GPIO access (RPi.GPIO does not work on the Pi 5).
- Query your time-series database and represent the result with another library or visualisation framework.
- Add simple AI at the edge, building on the last lab (e.g., anomaly detection on sensor data that triggers an actuator).

2.5 Variety of communication mechanisms

- REST over HTTP.
- Bluetooth, e.g., sending data via serial Bluetooth or Bluetooth Low Energy to a computer or mobile phone[5] (there are mobile apps to receive such data).
- MQTT (e.g., a Mosquitto broker[6] with paho-mqtt[7]), CoAP, etc. Note that some outbound ports, such as MQTT, are closed on the campus network, so run the broker locally (on the Raspberry Pi or your laptop) when working at the university.

2.6 Documentation

- Document your REST API with OpenAPI[8].
- Use GitHub, its README.md and its wiki[9] to document the project.

2.7 OS and Raspberry Pi resource management

- Use crontab or systemd services to manage your programs, and monitoring scripts started by the system.
- Report the status of the Raspberry Pi (CPU temperature, load).
- Use events, tasks or threads; store logs; interact with the OS through shell scripts.

2.8 Additional data sources and other inputs and outputs

- Interact with third-party APIs. Are there services that can provide relevant data for your project, such as weather and UV forecasts (Euskalmet), air quality (Open Data Euskadi) or Bilbao's open data on parks and green spaces? See the ideas in the IoT Challenge.

---

[2]Flask quickstart: https://flask.palletsprojects.com/en/stable/quickstart/
[3]Node-RED on Raspberry Pi: https://nodered.org/docs/getting-started/raspberrypi
[4]gpiozero documentation (works on the Raspberry Pi 5): https://gpiozero.readthedocs.io/
[5]Sending sensor data over serial Bluetooth: https://towardsdatascience.com/sending-data-from-a-raspberry-pi-sensor-unit-over-serial-bluetooth-f9063f3447af
[6]Mosquitto broker: https://mosquitto.org/
[7]MQTT in Python (paho-mqtt): https://www.hivemq.com/blog/implementing-mqtt-in-python/
[8]OpenAPI specification: https://www.openapis.org/
[9]GitHub wikis: https://docs.github.com/en/communities/documenting-your-project-with-wikis/about-wikis

- Use open maps to show where data was collected or where the action takes place: OpenStreetMap with Leaflet[10], etc. This fits this year's theme particularly well.

2.9 Additional functionalities (just some ideas)

- Python libraries that add new functionality.
- Telegram bots[11], emails or push notifications[12] for alerts.
- Voice command recognition[13] or text-to-speech with a loudspeaker[14].

> **An example of an excellent project (score: 10).** The project has three parts: a back end, a front end and a sensor script. The back end contains the API, which handles user management, sensor data queries and requests to an external service (e.g., a weather API). The front end displays the data from the REST API you created, including a map of the green spaces involved. The sensor script on the Raspberry Pi reads the sensors, uploads the data to InfluxDB and triggers local feedback (LEDs, buzzer, display) when conditions are met.

This does not mean that you must all do exactly that. Think about how to add elements that differentiate your proposal and bring it closer to your initial expectations. I will meet each group periodically to review progress and discuss implementation ideas.

3. Project management: GitHub and SCRUM

You will develop the project following the SCRUM methodology up to the MVP, and you will manage it entirely on GitHub.

1. **Create the repository today (3 November 2026).** One repository per group, with every member contributing from their own GitHub account. Add me as a collaborator (**dieguich**) on the same day.
2. **Track the project in GitHub Projects.** Keep a board with the product backlog (user stories or tasks), each sprint's backlog and the status of every item (To do / In progress / Done). Use issues for the items, assign each one to a member, and reference the issue in your commits and pull requests (e.g., `Closes #12`).
3. **Work in sprints up to the MVP (11 December 2026).** There are two Sprint Reviews with me, plus the final delivery:

Sprint	Dates	Goal / checkpoint
Sprint 1	3 Nov → 17 Nov	Sprint Review 1 (17 Nov): everything under control towards the MVP. Repository and board set up, product backlog with prioritised user stories, architecture defined, sensors and actuators tested on the Raspberry Pi, plan for the next sprints.
Sprint 2	17 Nov → 3 Dec	Sprint Review 2 (3 Dec): partial functionality working, at least one complete flow from sensor to storage/visualisation, shown live.
Sprint 3	3 Dec → 11 Dec	MVP delivered through ALUD (11 Dec). Tag the delivered version as a release (e.g., v1.0).
Demonstration	Week of 14 Dec	In-person presentation and demonstration of the MVP.

[10] Leaflet: https://leafletjs.com/
[11] Telegram bots: https://core.telegram.org/bots/tutorial
[12] Push notifications with ntfy: https://ntfy.sh/
[13] Offline speech recognition with Vosk: https://alphacephei.com/vosk/
[14] Text-to-speech with espeak-ng: https://github.com/espeak-ng/espeak-ng

Format. The Sprint Reviews and the final demonstration are held in person, group by group: 5 minutes per group for each Sprint Review and 15 minutes for the final demonstration of the MVP. No slides are needed: show the working system, the repository and the project board.

> **Continuous work.** I will follow your progress through the commit history and the project board. I expect regular commits from every member throughout the sprints, not a concentration of work in the last week. Each member must commit from their own account; if you work in pairs on one computer, credit your partner with a `Co-authored-by:` line in the commit message.

Good practices

- Use branches and pull requests, and have a teammate review them before merging.
- Never commit passwords, tokens or API keys. Keep them in a `.env` file listed in `.gitignore`; remember that the repository will be public.
- Commits after the release tag of 11 December will not be considered for the evaluation.

4. Submission and presentation

The MVP must be submitted through ALUD by **Friday, 11 December 2026**. The in-person demonstrations will take place during the week of 14 December 2026, group by group (15 minutes per group); I will agree a slot with each group.

4.1 Through ALUD (by 11 December 2026)

A zip file named `groupXX_Final_Project_IoT.zip` containing:

1. A TXT file with the link to your **public GitHub repository** and to its GitHub Projects board (mandatory). The repository must contain the source code and the functional description of the project in README.md and the wiki: the sensors and actuators it uses, their connections, what the system does and how the code is structured.
2. A **half-page joint reflection** on what you have learned in the course through the project-based approach, and a self-assessment: how each of you evaluates your own contribution and the work you have done together. Translate it into the grade you think you should get, based on your performance throughout the course and on this work in particular.

4.2 Demonstration (week of 14 December 2026)

In 15 minutes, show me the working prototype, give a demonstration and explain how you developed it. No slides are needed: work directly with the system, the code and the repository. All members of the group must take part and be able to answer questions about any part of the project.

5. Evaluation

This work accounts for 65% of the final grade. The grade will reflect the complexity of your solution, the concepts of the subject you have used and extended, and how well everything works. It will be based on the code, the documentation and your ability to “defend” the project. The commit history, the project board and the two Sprint Reviews will be used to assess continuous work and the contribution of each member.

The indicators are:

- **Originality:** What is the interest of the concept or outcome? Does it offer a unique approach or an original perspective (e.g., in the use of data or sensors)?

- **Scope:** Does it integrate the concepts seen in the course, build on them, or go beyond standard approaches?
- **Code:** Is the code well commented, well formatted, well structured, working and understandable?
- **Documentation:** Is it well written and detailed enough to replicate the result? Does it present the project and its objectives clearly?
- **Presentation:** Is the objective conveyed? Is the solution presented clearly and in an orderly way, with an explanation of how it works?
- **Content mastery:** Do all team members master what is presented? Do they understand the development and its outcome?

Rubric

Block	Indicator	Fail (0–4)	Pass (5–6)	Very good (7–8)	Outstanding (9–10)
Content	**Originality**	No originality or creativity (e.g., direct replication of a previous approach without extension or modification).	A small increment on existing work (extension or adaptation of a precedent).	Uses precedents and/or materials in a relatively original way.	Shows deep insight into the problem and significant originality in the ideas and their application.
	Scope	The contribution is insufficient for the expected level.	Minimal contribution; very limited use of the resources and content of the subject.	Appropriate use and combination of the resources and content of the subject.	Goes the extra mile, implementing complex functionality and showing technical competence.
Development	**Code**	The code is not included or does not work.	The code is messy or hard to understand.	Understandable, reasonably well-structured implementation, with comments that make it easy to follow.	Excellently organised implementation, commented in depth.
	Documentation	The documentation does not exist.	Contains the required components but is of low quality.	Detailed enough to reproduce the result easily (e.g., it details the third-party tools used).	Adds rigour or professionalism that elevates the outcome (e.g., ready to make the project public).
Presentation	**Project presentation**	Disordered speech with incoherent ideas; it is not clear what was done; nervousness prevents conveying the content.	Orderly but unclear speech; some parts better presented than others; sticks to time; synthesis can improve.	Fluent communication; sticks to time, though some parts and the synthesis can improve.	Fluent, clear and orderly communication; good synthesis; sticks to time.
	Content mastery	Does not answer questions correctly or show knowledge of the work.	Mechanical or read speech; answers questions but repeats the speech or is evasive.	Rarely relies on notes; answers questions adequately.	Fluent explanations; answers questions with a good command of the subject.